

Lab 6: Implementing the API Gateway Pattern (Using Nginx)

This lab focuses on implementing the API Gateway Pattern, a core component in Microservices Architecture. The Gateway acts as the single-entry point for all client requests (from the React Frontend), routing them to the appropriate Backend services (such as the Auth Service and Leave Service) and handling cross-cutting concerns like CORS, internal IP hiding, and Load Balancing.

Objectives

- 1. Understand the role of the API Gateway in a distributed Microservices environment.
- 2. Set up Nginx as a Reverse Proxy to coordinate network traffic.
- 3. Configure the Gateway to route HTTP Requests from the Frontend to the correct internal Microservices based on the URL path.
- 4. Test the routing capabilities and error handling (Service Unavailable) of the Gateway.

Tool & Technology Setup

In the actual project environment, we will use Nginx combined with Docker Compose to easily manage the internal network between containers.

Tool	Purpose	Installation/Setup Guide
Docker & Docker Compose	Manage and run Microservices alongside Nginx within a shared internal network.	Download and install Docker Desktop from the official website.
Nginx (Docker Image)	Acts as the API Gateway & Reverse Proxy.	No pre-installation required; Docker will pull the image automatically.
Postman / cURL	API testing tools.	Install Postman or use the curl command.

Activity Practice 1: Project Setup and Docker Compose

Goal: Create the configuration files to run Nginx and connect it to the Backend services (Auth, Leave).

Step-by-Step Instructions & Coding Guide:

1. **Create Gateway Directory:** Open the Terminal at the root of the project (same level as frontend, auth_service, leave_service):

```
bash
```

```
mkdir nginx_gateway
```

```
touch nginx_gateway/nginx.conf
```

```
touch docker-compose.yml
```

2. **Define Docker Compose (Microservices Network):** This configuration simulates a mini-ecosystem comprising the Nginx Gateway and 2 Microservices. Nginx will expose port 80 to the outside world, while the Backends remain hidden within the internal network. File: [docker-compose.yml](#)

```
yml
```

Show full code block

```
version: '3.8'
```

```
services:
```

```
# 1. API Gateway Service (Nginx)
```

```
api-gateway:
```

```
  image: nginx:alpine
```

```
  ports:
```

```
    - "80:80"
```

```
  volumes:
```

```
    - ./nginx_gateway/nginx.conf:/etc/nginx/nginx.conf
```

```
  depends_on:
```

```
    - auth-service
```

```
    - leave-service
```

```
  networks:
```

```
    - hrm-network
```

2. Identity Service (Auth Service)

auth-service:

build: ./auth_service # Requires a Dockerfile in this directory

expose:

- "5000"

networks:

- hrm-network

3. Leave Service (From Lab 5)

leave-service:

build: ./leave_service # Requires a Dockerfile in this directory

expose:

- "5002"

networks:

- hrm-network

networks:

hrm-network:

driver: bridge

Activity Practice 2: Implementing Routing Logic in Nginx

Goal: Configure Nginx to intercept requests on port 80 and route them based on the URL prefix.

Step-by-Step Instructions & Coding Guide:

1. **Write Routing Configuration (Routing Logic):** Open the [nginx.conf](#) file and add the configuration blocks. Notice how Nginx identifies `/api/auth_ser` and forwards it to the auth-service container. File: `nginx_gateway/nginx.conf`

nginx

Show full code block

```
events {  
    worker_connections 1024;  
}
```

```
http {  
    # Upstream Configuration (Points to service names in docker-compose)  
    upstream auth_backend {  
        server auth-service:5000;  
    }
```

```
    upstream leave_backend {  
        server leave-service:5002;  
    }
```

```
server {  
    listen 80;  
    server_name localhost;
```

1. CROSS-CUTTING CONCERN: Global CORS configuration for the whole system

```
    add_header 'Access-Control-Allow-Origin' '*';  
    add_header 'Access-Control-Allow-Methods' 'GET, POST, OPTIONS, PUT, DELETE';  
    add_header 'Access-Control-Allow-Headers' 'Authorization, Content-Type';
```

2. ROUTING LOGIC: Routing for Auth Service

```
location /api/auth_ser/ {
    proxy_pass http://auth_backend/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}
```

3. ROUTING LOGIC: Routing for Leave Service

```
location /api/leave_ser/ {
    proxy_pass http://leave_backend/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}
```

4. (Optional) Configuration for Socket.io (WebSocket support)

```
location /socket.io/ {
    proxy_pass http://auth_backend/socket.io/; # Assuming Auth Service is handling
    WebSockets
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "Upgrade";
}
}
```

Activity Practice 3: Testing the Gateway

Goal: Verify that the API Gateway correctly routes requests and handles service failures gracefully.

Prerequisites: Ensure you have a simple [Dockerfile](#) created in both the `auth_service` and `leave_service` directories.

Step-by-Step Instructions (Using cURL or Postman):

1. Start the System via Docker Compose:

- Action: Open the Terminal in the directory containing the [docker-compose.yml](#) file and run the system in detached mode.
- Command: `docker-compose up -d`

2. Test Successful Routing (Authorized Access):

- Action: Send a request to fetch the user list. Instead of calling port 5000 directly, call port 80 on the Gateway.
- cURL Command: `curl -X GET -H "Authorization: Bearer <valid_token>" http://localhost:80/api/auth_ser/users`
- Expected Result: HTTP 200 OK code and the user list (Request successfully routed by the Gateway to the Auth Service).

3. Test Routing to Another Service:

- Action: Send a request to count the number of absent employees today.
- cURL Command: `curl -X GET http://localhost:80/api/leave_ser/stats/today`
- Expected Result: HTTP 200 OK code returning `{"count": 0}` (Gateway automatically detects the `/api/leave_ser/` prefix and routes to port 5002).

4. Test Service Failure Handling:

- Action: Simulate a failure by stopping the Leave Service container.
- Command: `docker stop <leave_service_container_name>`
- Action 2: Attempt to call the absent employee count API from step 3 again.
- cURL Command: `curl -X GET http://localhost:80/api/leave_ser/stats/today`
- Expected Result: Nginx will return an **HTTP 502 Bad Gateway** error. This protects the client from hanging indefinitely when a backend crashes, proving the power of the API Gateway.